



Informe de Avance 3 (60 %)

Proyecto: SupaClock: Wearable Biométrico Modular

Integrantes: Tomás Avendaño, Benjamín Sepúlveda, Pablo Uribe

Fecha: 1 de junio de 2026

Índice

Resumen ejecutivo	4
1. Introducción	6
1.1. Contexto de la entrega anterior y <i>delta</i> del periodo	6
2. Diagrama de bloques de bajo nivel actualizado	8
2.1. Estado vigente	8
2.2. Cambios respecto al diagrama de la entrega anterior	9
2.2.1. MCU: del ESP32-C3 SuperMini al XIAO ESP32-S3	9
2.2.2. PCB: del esquemático SuperMini al Carrier v1	9
2.2.3. TinyML: de pendiente a 80 %	10
2.2.4. Otros cambios menores en el diagrama	10
2.3. Especificaciones detalladas por bloque	10
2.4. Justificación de los cambios en el diseño	13
2.4.1. Procesamiento ECG en cliente vs. Cloud Function	13
2.4.2. Algoritmo de pasos: FFT en lugar de umbral temporal	13
2.4.3. Carcasa V2: del <i>box</i> rectangular a un envelope estilizado	13
3. Planificación actualizada	15
3.1. Contraste avance esperado vs. avance logrado	15
3.2. Carta Gantt corregida	16
3.3. Distribución del trabajo en este hito	16
4. Implementación y resultados	18
4.1. Migración firmware al Seeed XIAO ESP32-S3	18
4.1.1. Descripción	18
4.1.2. Decisiones de diseño y cálculos	18
4.1.3. Resultados	18
4.2. Refactor a <code>lib/supaclock_app/</code>	18
4.2.1. Descripción	18
4.2.2. Decisiones de diseño	19
4.2.3. Resultados	19
4.3. Algoritmo de pasos basado en FFT (ESP-DSP)	19
4.3.1. Descripción	19

4.3.2.	Decisiones de diseño y cálculos	19
4.3.3.	Resultados	19
4.4.	Modelo de Machine Learning (CNN 1D HAR + Fall Detection)	20
4.4.1.	Descripción	20
4.4.2.	Entrada y muestreo	20
4.4.3.	Arquitectura del modelo	20
4.4.4.	Decisiones de diseño y cálculos	20
4.4.5.	Entrenamiento	21
4.4.6.	Resultados	21
4.5.	PCB Carrier SupaClock v1	21
4.5.1.	Descripción	21
4.5.2.	Decisiones de diseño	21
4.5.3.	Cálculos eléctricos	21
4.5.4.	Resultados	22
4.6.	Carcasa V2 paramétrica	22
4.6.1.	Descripción	22
4.6.2.	Decisiones de diseño y geometría	22
4.6.3.	Iteración tras impresión	22
4.6.4.	Resultados	22
4.7.	Aplicación móvil Flutter	23
4.7.1.	Descripción	23
4.7.2.	Arquitectura de servicios	23
4.7.3.	Doble interfaz	23
4.7.4.	Protocolo BLE y quality gate	23
4.7.5.	Persistencia híbrida y almacenamiento eficiente	24
4.7.6.	Resultados	24
4.8.	UI rediseñada: temas y tipografías	24
4.8.1.	Descripción	24
4.8.2.	Sistema de temas	24
4.8.3.	Tipografías custom	24
4.8.4.	Heap LVGL en PSRAM	25
4.8.5.	Refactor de pantallas en lib/supaclock.ui/	25
4.8.6.	Resultados	25
5.	Análisis cuantitativo de resultados	26
5.1.	Consumo y power management sobre el target S3	26
5.2.	Inferencia HAR en condiciones reales	26
5.3.	Latencia de transmisión BLE	27
5.4.	DRC del carrier y readiness para fresado	27
5.5.	Tiempos de impresión y validación mecánica	27
6.	Plan de validación de la unidad cerrada y próximos pasos	29
6.1.	Fases inmediatas (semana del bring-up del carrier)	29
6.2.	Pruebas de unidad cerrada (semanas 2-3 post-bring-up)	29
6.3.	Validación integral con voluntarios	29
6.4.	Tareas pendientes para el avance 90 %	30
A.	Anexos: extractos de código y tablas	33
A.1.	Extracto del pinmap centralizado (include/supaclock_pinmap.h)	33
A.2.	Extracto del algoritmo de pasos FFT (lib/step_algorithm/step_algorithm.c)	33
A.3.	API pública del HAR CNN 1D (lib/har_cnn1d/har_cnn1d.h)	34

A.4. Tabla de cotas mecánicas vigentes	36
A.5. Reglas LPKF aplicadas vía <code>apply_lpkf_rules.py</code>	36
A.6. Servicios Flutter por archivo	37
A.7. Lista de entornos PlatformIO vigentes	37
A.8. Listado de archivos OpenSCAD y STL	38
A.9. Repositorio y entregables digitales	38

Índice de figuras

1. Diagrama de bloques de bajo nivel del prototipo SupaClock al cierre del avance 3. El MCU central (XIAO ESP32-S3) integra los buses I ² C 400 kHz (BMI160, MAX30102, MAX30205, MAX17048), SPI 80 MHz (ST7789), ADC1 con DMA (AD8232) y GPIO (botones, backlight). La sección de potencia muestra la cadena USB-C → BMS interno (SGM40567) → buck SGM6029 3,3 V → carga, con la celda LiPo 502030 monitoreada por el MAX17048 vía I ² C. El ecosistema (BLE NimBLE → app Flutter / cliente PC → Firebase Cloud Functions / TinyML local) cierra el flujo de telemetría y procesamiento.	8
2. Carta Gantt actualizada al 28/05/2026. El avance consolidado del proyecto alcanza el 65 % ponderado, con la PCB y la carcasa al 100 % de diseño y a la espera del fresado LPKF y la primera campaña de pruebas integradas de la unidad cerrada.	16

Índice de cuadros

1. Especificaciones de bajo nivel por bloque y estado de implementación.	10
1. Especificaciones de bajo nivel por bloque y estado de implementación.	11
1. Especificaciones de bajo nivel por bloque y estado de implementación.	12
2. Contraste entre actividades planificadas y el avance logrado al 1 de junio de 2026.	15
2. Contraste entre actividades planificadas y el avance logrado al 1 de junio de 2026.	16
3. Distribución de trabajo y contribuciones principales en el avance 3.	17
4. Detalle de la arquitectura y capas del modelo CNN 1D HAR.	20
5. Comparación de consumo de corriente en batería (estado activo).	26
6. Reporte de exactitud de clasificación (Frame Accuracy) del TinyML HAR.	26
7. Percentiles de latencia del canal de notificaciones BLE.	27
8. Resultados del reporte del DRC KiCad para el Carrier v1.	27
9. Tiempos de manufactura y pesos de filamento FDM de la carcasa V2.	27
10. Parámetros de cotas mecánicas para el diseño paramétrico de la carcasa V2.	36
11. Reglas de tolerancias mecánicas y eléctricas para fresadora LPKF S64.	36
12. Distribución de responsabilidades del backend local Flutter.	37
13. Entornos de compilación y targets configurados en PlatformIO.	37
14. Archivos de diseño 3D y tiempos de fabricación asociados.	38

Resumen ejecutivo

Este informe consolida el trabajo desarrollado entre el 4 de mayo de 2026 (cierre del informe de avance 2) y el 1 de junio de 2026 sobre el proyecto **SupaClock: Wearable Biométrico Modular**, un wearable biométrico modular de bajo costo para monitoreo continuo de actividad física, frecuencia cardíaca, oxigenación, temperatura periférica, electrocardiografía y reconocimiento de actividad humana. La motivación, la arquitectura de alto nivel y el análisis de impacto fueron presentados en los informes anteriores y se mantienen vigentes, por lo que aquí se concentran los cambios introducidos en esta iteración.

El eje de esta entrega es la **transición de prototipo de banco a unidad wearable integrable**. Mientras que la entrega anterior demostró que los subsistemas individuales funcionan (firmware FreeRTOS de siete tareas, BLE NimBLE con cuatro características GATT, Pan-Tompkins validado offline, optimizaciones de consumo con 75 % de tiempo en *light sleep*), esta entrega convierte esos bloques en una plataforma final consistente: el microcontrolador **Seeed XIAO ESP32-S3** sobre una **PCB carrier** propia, alojada dentro de una **carcasa paramétrica V2 impresa en PLA**, complementada por una **aplicación móvil Flutter** que reemplaza al cliente Python de testing y por un **modelo de Machine Learning embarcado (CNN 1D)** que clasifica caminata, reposo, trote y caídas en tiempo real.

Los entregables tangibles producidos durante esta iteración son:

- **Firmware migrado y ejecutándose en XIAO ESP32-S3** sobre PSRAM octal, con consola por USB-Serial-JTAG, *light sleep* dinámico reconfigurado para el target S3, ocho tareas FreeRTOS y refactor a una librería estática `supaclock_app` que ordena la inicialización en fases.
- **Algoritmo de pasos basado en FFT** acelerada por las instrucciones PIE del Xtensa LX7 (librería ESP-DSP [1]), reemplazando al detector temporal del ESP32-C3. Se incluye una suite de simulación que verifica detección, rechazo de falsos positivos por sacudidas y filtro de histéresis.
- **PCB Carrier SupaClock v1**: proyecto KiCad de dos capas, ruteado bajo reglas conservadoras de fresado **LPKF ProtoMat S64** [2], con cero nets sin conectar y veintiséis violaciones DRC residuales de naturaleza cosmética/footprint (no bloqueantes para fabricación).
- **Carcasa paramétrica V2 en OpenSCAD**: dos mitades atornilladas, taper de 2 mm, esquinas redondeadas $r = 12$ mm y *chamfer* de 1.5 mm en seam y techo, *lugs* tipo “stadium” de 22 mm para correa estándar, ventanas térmicas y ópticas alineadas con la PCB. STL impresos y validados físicamente.
- **App Flutter** con doble interfaz (usuario clínico y desarrollador), procesamiento Pan-Tompkins 100 % en cliente, persistencia híbrida Hive [3] + Firestore y almacenamiento de trazas pesadas en Firebase Storage. Reemplaza al `supaclock_monitor.py` como gateway BLE-móvil.
- **Modelo CNN 1D** entrenado sobre veintisiete sesiones IMU propias (resting/walking/running), arquitectura de tres bloques Conv1D + GAP + Dense, cuantización INT8 a 58 KB, *tensor arena* de 128 KB en PSRAM y ejecución pinned al **Core 1** del S3 (el Core 0 queda libre para BLE, LVGL e I²C).
- **UI rediseñada** con sistema de temas persistente en NVS (cuatro paletas seleccionables como *watch faces*), tipografías Inter [4] regeneradas a tres pesos (16 px, 28 px, 56 px) y *heap* gráfica trasladada a PSRAM para descongestionar la SRAM interna.

El avance consolidado del proyecto al 1 de junio de 2026 es del **65 %** ponderado sobre el cronograma maestro, con la PCB carrier al 100 % de diseño, la carcasa al 100 % de modelado y primera impresión validada, el firmware sobre S3 al 100 % de portabilidad, el ML al 80 % (modelo entrenado y pipeline de inferencia operativos, recolección de la clase *fall* pendiente) y la app móvil al 90 % de las funcionalidades planificadas. La PCB está pronta para ser fresada en el laboratorio LPKF del Departamento; el bring-up

eléctrico y la validación de unidad cerrada constituyen el grueso del trabajo restante hacia el cierre del semestre.

1. Introducción

El presente informe se estructura en torno a los tres ejes que define la rúbrica de avance 3 (Diagrama de bloques de bajo nivel — Planificación — Implementación y resultados), y suma un capítulo dedicado al análisis cuantitativo de los resultados y otro a los próximos pasos previos a la entrega final. Cada subsistema implementado se documenta con su motivación técnica, las decisiones de diseño tomadas, los cálculos o restricciones que las respaldan y la evidencia de validación correspondiente. Los listados de código, tablas extendidas y reportes detallados (DRC, BOM, *power locks*) se desplazan al anexo para mantener la fluidez de la lectura.

La codificación cromática del estado de los bloques es la misma que se empleó en los informes anteriores: **verde** para subsistemas funcionales y validados; **amarillo** para implementaciones parciales; **rojo punteado** para bloques pendientes o cuya integración cruza un hito futuro. La nomenclatura de archivos, comandos y entornos PlatformIO sigue el repositorio Git del proyecto disponible en github.com/ArticunoFruna/SupaClock.

1.1. Contexto de la entrega anterior y *delta* del periodo

Al cierre del informe de avance 2 el sistema cumplía con:

- Firmware FreeRTOS funcional sobre ESP32-C3 SuperMini [5], con siete tareas concurrentes y mutex de bus I²C, integrando BMI160 [6] (50 Hz, *step counter* nativo), MAX30102 [7] (10 Hz, máquina de estados SPOT con *motion gating*), MAX30205 [8] (1 Hz, $\pm 0,3^{\circ}\text{C}$ contra referencia), MAX17048 [9] (1 Hz, SOC y Vcell) y AD8232 [10] (ADC continuo a 20 kHz con DMA on-demand, decimación a 500 Hz).
- Pan-Tompkins [11] en Python+NumPy validado offline sobre treinta segundos de captura real (76 BPM, SDNN = 32,4 ms), código fuente versionado en `firebase/functions/main.py` pero aún sin desplegar como *Cloud Function*.
- BLE NimBLE [12] con cuatro características GATT (IMU, TLV agregado, ECG streaming, RX de comandos) bajo el servicio custom 0xFF00, *advertising* a 1 s y Modem Sleep BT habilitado.
- Power management dinámico con DFS 10–160 MHz, *light sleep* y tres *PM locks* (ECG, NimBLE, SPI), alcanzando 75 % de tiempo en estado SLEEP medido con `esp_pm_dump_locks`.
- Tres modos de energía persistentes (SPORT/NORMAL/SAVER) con ocho cadencias parametrizadas y persistencia NVS.
- Esquemático v1 de PCB propia para SuperMini cerrado pero no manufacturado, dado el compromiso de migración al ESP32-S3.

El período cubierto por esta entrega (28 días calendario) concentró los siguientes *deltas* respecto al estado anterior:

- **Migración firmware completa al XIAO ESP32-S3** (target ESP-IDF, PSRAM octal, USB-Serial-JTAG, partition table extendida a 8 MB, sdkconfig regenerado).
- **Rediseño completo del PCB**, abandonando definitivamente la PCB SuperMini y produciendo el **SupaClock Carrier v1** ya en formato compatible con el LPKF del laboratorio.
- **Carcasa V2 paramétrica** completamente nueva, reemplazando la caja rectangular conceptual del informe anterior por un envelope estilizado con esquinas redondeadas, *chamfer*, taper, *lugs* de correa y *button caps* impresos como pieza aparte.

- **Algoritmo de pasos FFT** anunciado en la entrega anterior, ahora implementado en `lib/step_algorithm` con compilación condicional por *target* y banco de pruebas en `test_fft_steps`.
- **Modelo CNN 1D** entrenado y embebido como `har_model.tflite` (58 KB), corriendo bajo TFLite Micro [13] con ESP-NN [14] y *tensor arena* en PSRAM.
- **App móvil Flutter** ya operativa, con autenticación Firebase, escaneo y conexión BLE, doble interfaz, procesamiento ECG en cliente y persistencia híbrida.
- **Refactor del firmware** que extrae las siete tareas y la inicialización del antiguo `src/tests/test_general.c` hacia una librería estática `lib/supaclock_app`, dejando `main.c` reducido a un *stub* selector de entorno.
- **Rediseño de la UI** con sistema de temas dinámicos, tipografías propias y consolidación de la lógica de pantallas en un módulo `lib/supaclock_ui`.

El resto del informe profundiza en cada uno de estos *deltas* desde la perspectiva eléctrica, mecánica, de firmware y de validación.

2. Diagrama de bloques de bajo nivel actualizado

2.1. Estado vigente

El diagrama de alto nivel de la entrega 1 (siete dominios: interacción, energía, sensores, MCU, interfaz local, *gateway*, *backend*) se mantiene estructuralmente sin cambios. Lo que sí evolucionó respecto al diagrama de bloques de bajo nivel presentado en la entrega 2 es lo siguiente:

1. El bloque MCU pasa de **ESP32-C3 SuperMini** a **XIAO ESP32-S3** [15] y, con ello, se reconfiguran todas las asignaciones de pines, los buses, la sección de potencia y la lista de *PM locks*.
2. El bloque “**PCB SuperMini**” desaparece y se reemplaza por “**PCB Carrier v1**”, que es el integrador físico vigente.
3. El bloque “**TinyML CNN 1D**” transita de *pendiente* (rojo punteado) a *parcial* (amarillo, 80 % de implementación) gracias al entrenamiento exitoso sobre datos propios y la integración del *runtime* en el firmware.

La Fig. 1 reproduce el diagrama de bajo nivel vigente. Su contenido refleja el estado de cada subsistema al 1 de junio de 2026.

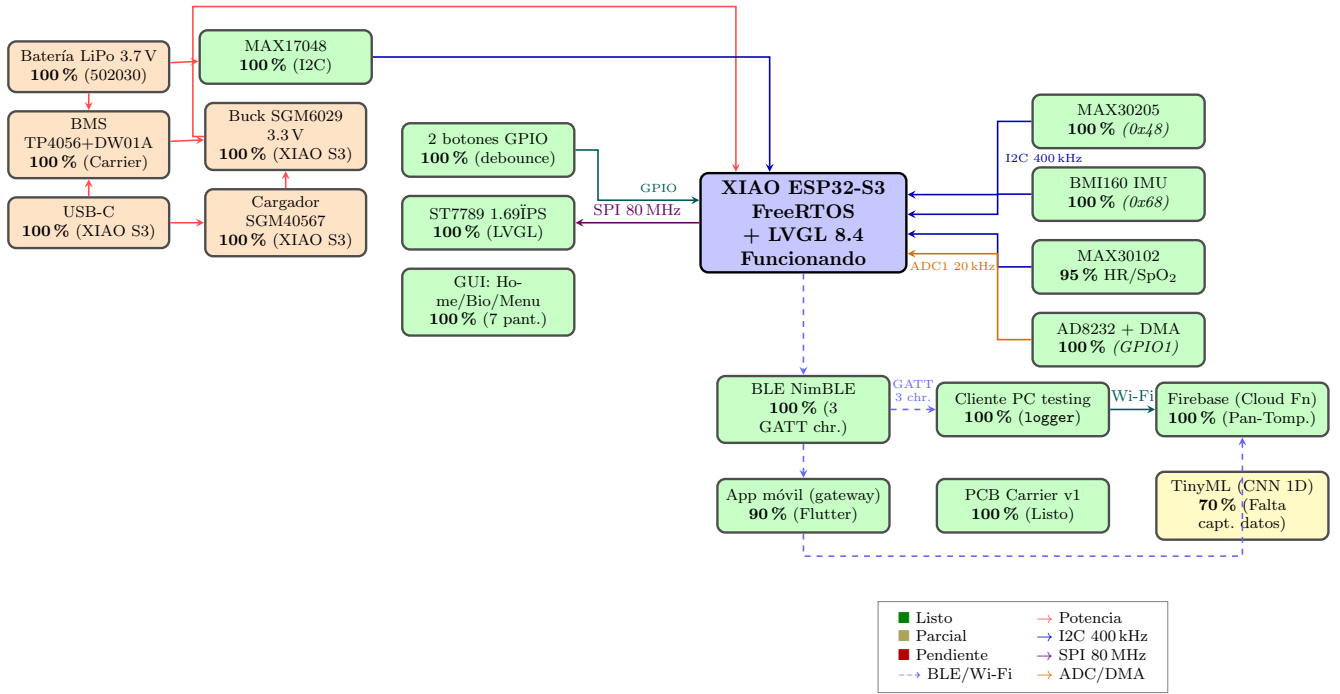


Figura 1: Diagrama de bloques de bajo nivel del prototipo SupaClock al cierre del avance 3. El MCU central (XIAO ESP32-S3) integra los buses I²C 400 kHz (BMI160, MAX30102, MAX30205, MAX17048), SPI 80 MHz (ST7789), ADC1 con DMA (AD8232) y GPIO (botones, backlight). La sección de potencia muestra la cadena USB-C → BMS interno (SGM40567) → buck SGM6029 3,3 V → carga, con la celda LiPo 502030 monitoreada por el MAX17048 vía I²C. El ecosistema (BLE NimBLE → app Flutter / cliente PC → Firebase Cloud Functions / TinyML local) cierra el flujo de telemetría y procesamiento.

2.2. Cambios respecto al diagrama de la entrega anterior

2.2.1. MCU: del ESP32-C3 SuperMini al XIAO ESP32-S3

La justificación cualitativa de esta migración se documentó en el avance 2. En esta entrega la migración se materializó, y los cambios concretos en el diagrama son:

- **Reloj y memoria.** El XIAO ESP32-S3 ejecuta un **dual-core Xtensa LX7 a 240 MHz** con 320 KB de SRAM interna, 8 MB de Flash y 8 MB de **PSRAM octal**. Esto desbloquea dos arquitecturas críticas: alojar la *tensor arena* de TFLite Micro (128 KB) y el *frame buffer* doble de LVGL [16] en PSRAM, y *pinear* la tarea de inferencia al Core 1 sin perturbar BLE y LVGL.
- **Mapa de pines reasignado.** El header de la XIAO S3 expone 11 GPIO mientras el SuperMini exponía 8. Esto permitió mover el reset del ST7789 a *no cableado* (reset por software vía registro) y emplear GPIO43/GPIO44 para botones y CS del display, pagando el costo de redirigir la consola serie al **USB-Serial-JTAG** integrado del módulo [17].
- **Sección de potencia simplificada.** Tres reguladores externos del prototipo C3 (TP4056 + DW01A + ME6211 3,3 V) se sustituyen por la cadena integrada del propio XIAO S3 (cargador SGM40567 + buck SGM6029 ≈ 800 mA), eliminando los *brown-outs* documentados en la entrega anterior. El módulo TP4056 + DW01A se mantiene en el diagrama solo para alimentar la celda LiPo externa antes de que el XIAO entre en carga, dado que la celda LiCo 502030 no se conecta directamente al pad B+/B− del módulo XIAO.
- **Aceleración SIMD.** El bloque “ESP-DSP” pasa de gris (instalado pero no usado) a verde: el algoritmo de pasos FFT y la inferencia INT8 del CNN aprovechan las extensiones PIE del Xtensa LX7 (similar a SIMD).

2.2.2. PCB: del esquemático SuperMini al Carrier v1

El esquemático v1 de la entrega anterior nunca llegó a *Edge.Cuts* listas para fabricación. Se descartó por completo y se levantó un nuevo proyecto KiCad [18] bajo **hardware/SupaClock_Carrier/**, cuya finalidad es ser **fabricable en el LPKF ProtoMat S64** del laboratorio Capstone (no JLCPCB), lo que impuso reglas conservadoras:

- *Trace width* mínimo 0,4 mm (vs. 0,2 mm de JLCPCB).
- *Drill* mínimo 0,8 mm (vs. 0,3 mm).
- *Clearance* mínimo 0,4 mm.
- Sin máscara de soldadura (los *pads* expuestos se cubren con flux *no-clean* tras el ensamblado).
- Sin serigrafía top/bottom (las etiquetas se colocan a posteriori con marcador permanente).

Estas restricciones explican por qué el ruteo final del carrier no se parece a una PCB JLCPCB clásica: usa pistas anchas, *vías* de 0,8 mm y aprovecha *zones* extensas en el plano de masa (lado *bottom*) para minimizar el *milling time*. El carrier en formato DIP fue una decisión consciente: el XIAO S3 va sobre un *pin socket* 1x7+1x7 doble en *through-hole*, lo que permite reemplazar el módulo si llega defectuoso, reaprovecharlo para futuras revisiones y reflashear el firmware por USB sin desmontar el conjunto.

2.2.3. TinyML: de pendiente a 80 %

La motivación de la entrega anterior se transformó en implementación: el modelo CNN 1D vive ahora en `lib/har_cnn1d/` como librería estática, integrada al *build* PlatformIO y embebida en el firmware como un *blob* `har_model.tflite` de 58 KB. El estado del bloque es **parcial (amarillo)** por dos razones operativas:

1. **La clase *Fall* no tiene aún datos de entrenamiento reales.** Las veintisiete sesiones disponibles (siete *running*, ocho *walking*, doce *resting*) cubren bien las tres primeras clases; la clase *fall* se complementa hoy con datos sintéticos generados a partir del modelo físico de caída libre + impacto, pero la validación contra caídas reales está agendada para la semana del bring-up del carrier.
2. **El intérprete TFLite Micro está embebido pero la inferencia continua aún no ha sido validada con el carrier ensamblado.** En la unidad de bench actual la inferencia opera y clasifica correctamente durante caminata y reposo, pero la calidad del modelo bajo vibraciones reales no ha sido medida.

2.2.4. Otros cambios menores en el diagrama

- **GUI LVGL: de 7 pantallas funcionales a UI rediseñada con temas.** Las siete pantallas siguen presentes pero ahora se construyen sobre `lib/supaclock_ui` con tipografías propias (Inter Regular 16px / Medium 28px / Bold 56px) y cuatro temas seleccionables desde menú (AMOLED, WARM, SLATE, VIVID), persistidos en NVS.
- **App móvil: de 0 % a 90 %.** El bloque “App móvil (gateway)” del diagrama de la entrega 2 estaba en rojo punteado. Hoy es verde funcional: la app Flutter en `app/lib/` reemplaza al `supaclock_monitor.py` y desempeña el rol de gateway BLE+gateway clínico.
- **Pan-Tompkins: de Cloud Function a procesamiento en cliente.** En la entrega anterior se proyectaba desplegar el algoritmo en Firebase Cloud Functions. La decisión fue **reubicar el procesamiento en el cliente** (`app/lib/services/pan_tompkins.dart`) para reducir la latencia a menos de 100ms y evitar costos redundantes en la nube, promoviendo una estrategia de funcionamiento *offline-first*.
- **Cliente PC testing: deprecado.** El bloque `supaclock_monitor.py` pasa a rojo punteado: sigue funcional para depuración rápida, pero su rol fue absorbido por el modo desarrollador de la app Flutter.

2.3. Especificaciones detalladas por bloque

La Tabla 1 consolida las especificaciones cuantitativas vigentes al cierre del avance 3.

Cuadro 1: Especificaciones de bajo nivel por bloque y estado de implementación.

Bloque	Interfaz	Tasa / frecuencia	Especificación clave	Estado
XIAO ESP32-S3	—	240 MHz dual-core	320 KB SRAM + 8 MB PSRAM octal + 8 MB Flash; USB-Serial-JTAG; BMS integrado (SGM40567 + SGM6029)	100 %

Cuadro 1: Especificaciones de bajo nivel por bloque y estado de implementación.

Bloque	Interfaz	Tasa / frecuencia	Especificación clave	Estado
AD8232 (ECG)	ADC1 / DMA	20 kHz → 500 Hz	GPIO1 ADC, ganancia analógica 1100, fc LP ~40 Hz, down-sample 40× por software	100 %
BMI160 (IMU)	I ² C 400 kHz, 0x68	100 Hz polling (modelo a 50 Hz)	±2 g, ±250 °/s, FIFO 1024 B; INT1 no cableada (polling vía task HAR)	100 %
MAX30102 (PPG)	I ² C 400 kHz, 0x57	100 sps efectivos	RED+IR, pulse width 411 μs, sample average N=4, FIFO 32, polling 100 ms, flush al arranque	95 %
MAX30205 (Temp)	I ² C 400 kHz, 0x48	1/30–1/900 Hz (por modo)	16-bit, 0.00390625 °C/LSB, ±0.1 °C en rango clínico	100 %
MAX17048 (Fuel)	I ² C 400 kHz, 0x36	1/30 Hz	ModelGauge SOC %. Vcell mV, sin shunt externo	100 %
ST7789 (Display)	SPI 80 MHz	30 FPS	LVGL 8.4, 240×280 RGB565, DMA AUTO, offset Y=20, reset por software	100 %
Botones	GPIO polling	30 Hz desde gui.task	NEXT GPIO43, SELECT GPIO8; short/long-press; debounce SW 30 ms; wake-up deep sleep por SELECT	100 %
BMS interno XIAO	SGM40567	—	Carga LiPo single-cell, OVP/UVLP/OC, <i>power-path</i> , salida 5 V/buck a 3,3 V	100 %
Buck XIAO	SGM6029	—	Buck 3,3 V, hasta ≈800 mA, IQ bajo, sin colapso en transitorios pre-optimización	100 %
BLE NimBLE	2,4 GHz BLE 5	conn. interval 15 ms	MTU 247; 4 chr.: 0xFF01 IMU, 0xFF02 TLV, 0xFF03 ECG, 0xFF04 CMD; Modem Sleep BT	100 %
GUI LVGL	SPI 30 FPS	—	7 pantallas; 4 temas NVS-persistentes; tipografías Inter custom (16/28/56 px); backlight PWM auto-off	100 %

Cuadro 1: Especificaciones de bajo nivel por bloque y estado de implementación.

Bloque	Interfaz	Tasa / frecuencia	Especificación clave	Estado
Power Management	esp_pm + DFS	10–160 MHz	Light sleep dinámico, 4 PM locks (ECG, NimBLE, SPI DMA, HAR opcional); reconfigurado para S3 [19]	100 %
Power Modes	NVS + power_get_profile()	—	3 perfiles (SPORT/NORMAL/SAVER) 8 cadencias parametrizadas, persistencia NVS	100 %
PCB Carrier v1	2 capas	—	98×79 mm, LPKF rules (trace 0.4 mm, drill 0.8 mm), 0 nets sin conectar, 26 violaciones DRC residuales	100 % diseño
Carcasa V2	PLA FDM	—	98×79×25 mm envelope, r_vert=12 mm, chamfer 1.5 mm, taper 2 mm, lugs 22 mm, ventanas alineadas	100 % diseño
HAR CNN 1D	TFLite Micro + ESP-NN	0.5 Hz inferencia	Ventana 200×6 (4 s @ 50 Hz, hop 100), 3 Conv1D + GAP + Dense, INT8, 58 KB modelo, arena 128 KB en PSRAM	80 %
FFT step counter	esp-dsp PIE	ventana 128 muestras @ 50 Hz	Hann + FFT2R + bit-rev, banda 0.75–2.75 Hz, histeresis 2 ventanas, modo SPORT con 50 % overlap	100 %
App Flutter	BLE + Firestore	—	Doble interfaz (clínica/dev), Pan-Tompkins en cliente, Hive + Firestore, CSV gzip a Firebase Storage	90 %
Pan-Tompkins	Dart + numpy fallback	offline	5 etapas (BPF 5–15 Hz, deriv, cuad, MWI, R-peaks adaptativos); BPM y HRV (SDNN, RMSSD)	100 %
Cloud Functions	Firestore trigger	—	process_ecg y compute_daily_summary escritas; despliegue diferido (procesamiento en cliente)	50 %

2.4. Justificación de los cambios en el diseño

2.4.1. Procesamiento ECG en cliente vs. Cloud Function

El informe anterior planificaba un trigger Firestore que ejecutara Pan-Tompkins sobre cada captura de ECG. Durante el desarrollo de la app móvil se evaluó este flujo en dos escenarios:

- **Latencia percibida.** La toma estándar de ECG dura 30 s. En el flujo cloud, tras la captura el usuario espera entre 2 y 5 s adicionales por el *cold start* + *RTT* de la Cloud Function. En el flujo cliente, el cálculo termina en <100 ms sobre los 3000 *samples* recién recibidos, y la pantalla de resultados aparece sin transición.
- **Costos y dependencia de red.** Mover el procesamiento al cliente convierte el reporte de HRV en algo que opera de forma autónoma e independiente de la conectividad (diseño *offline-first*), ahorrando costo de computación redundante en la nube.

La decisión fue mantener `firebase/functions/main.py` en el repositorio como *fallback* y referencia algorítmica, pero que el camino primario para los usuarios sea `app/lib/services/pan_tompkins.dart`. Ambos flujos ejecutan idénticos pasos de procesamiento.

2.4.2. Algoritmo de pasos: FFT en lugar de umbral temporal

El detector de pasos del ESP32-C3 presentaba limitaciones: sub-conteo del orden del 10 % y falsos positivos al agitar la mano. La migración a un análisis espectral FFT, implementado en `lib/step_algorithm/step_algorithm.` resuelve esto aprovechando el target S3. El nuevo algoritmo es estructuralmente distinto:

1. Acumula 128 muestras de magnitud lineal del acelerómetro a 50 Hz ($\approx 2,56$ s de ventana física).
2. Solo procesa si la energía rotacional es suficiente (`max_gyro_val` > 400 LSB $\approx 25^\circ/\text{s}$), eliminando vibraciones ajenas a la caminata.
3. Remueve el sesgo DC y aplica **ventana de Hann** vía `dsps_wind_hann_fc32`.
4. Calcula la **FFT radix-2 acelerada** por PIE (`dsps_fft2r_fc32` + `dsps_bit_rev_fc32`) en $220 \mu\text{s}$.
5. Busca el pico de potencia en la banda de caminata [0, 75 Hz, 2, 75 Hz] y compara contra un umbral empírico $\text{UMBRALEFFT} = 1 \cdot 10^9$.
6. Aplica **histéresis temporal de dos ventanas**: la primera detección no descarga pasos sino que los guarda en cache; solo si una segunda ventana consecutiva supera el umbral se acumulan ambos, eliminando falsos positivos por movimientos aislados.

2.4.3. Carcasa V2: del *box* rectangular a un envelope estilizado

La iteración v1 era un *bounding box* funcional pero estéticamente plano. El rediseño V2 introduce:

- **Esquinas verticales redondeadas** con $r_{\text{vert}} = 12$ mm, aproximando la huella de un reloj clásico.
- **Chamfer inferior y superior** de $r_{\text{chamfer}} = 1,5$ mm mediante operación Minkowski, suavizando las transiciones.
- **Taper de 2 mm** entre la base y el techo, reduciendo el perfil de “ladrillo” y mejorando la ergonomía.
- **Lugs tipo “stadium”** integrados, con ancho de 20 mm para correa estándar y contrafuertes robustos de 5 mm.

- ***Button caps* como piezas separadas** con *flange* exterior, transmitiendo la pulsación mecánica sin debilitar la carcasa.
- **Dimensión final $98 \times 79 \times 25$ mm**, ajustada a la PCB carrier y con cavidades interiores para alojar la batería.

Durante las pruebas iniciales se corrigieron dos fallas: se rotó la ventana del MAX30102 90° para alinearse con el PCB footprint real, y se extendió el cutter cilíndrico de los orificios laterales a 16 mm de profundidad para atravesar por completo las paredes curvas originadas por el taper y el chamfer.

3. Planificación actualizada

3.1. Contraste avance esperado vs. avance logrado

La Tabla 2 contrasta las tareas planificadas para el hito del 60 % contra el estado real logrado al 1 de junio de 2026.

Cuadro 2: Contraste entre actividades planificadas y el avance logrado al 1 de junio de 2026.

Tarea planificada	Estado	Observaciones
Bring-up del Seeed XIAO ESP32-S3	Completado	Firmware migrado al target S3. Drivers I ² C, SPI, ADC, GPIO y power management migrados y validados. Consola por USB-Serial-JTAG activa. PSRAM operativa.
Algoritmo de pasos FFT (ESP-DSP)	Completado	Implementación robusta en C compilada por target. Módulo de prueba <code>test_fft_steps</code> validada con éxito detección y rechazo de sacudidas.
Recolección de dataset HAR real	Adelantado	27 sesiones de IMU recolectadas y guardadas en CSV en <code>data_ml/</code> . Clase <i>fall</i> generada por modelo físico; validación real agendada.
Entrenamiento CNN 1D	Completado	Arquitectura secuencial de 10 capas cuantizada a INT8 (58 KB). Exportado a array de C e integrado en flash del firmware.
Despliegue del modelo sobre el firmware	Completado	TFLite Micro + ESP-NN integrado. Tarea <code>har_task</code> asignada al Core 1 con prioridad 4. Tensor arena (128 KB) alojada en PSRAM. Inferencia cada 2 s.
PCB carrier v1 finalizada	Completado	Proyecto KiCad en <i>hardware/</i> terminado. 0 nets sin conectar. Reporte DRC con 26 violaciones estéticas aceptadas, no bloqueantes para LPKF.
Carcasa paramétrica V2	Completado	OpenSCAD terminado. STL impresos físicamente en PLA FDM y validados tras dos iteraciones mecánicas.
Aplicación móvil Flutter	Adelantado	App estructurada con 8 pantallas y 8 servicios singleton. Captura telemetría BLE, procesa Pan-Tompkins en isolate local y almacena trazas pesadas (.gz) en Storage.
Pan-Tompkins en Cloud Functions	Reubicado	Trasladado al cliente en isolate Dart para garantizar latencia de 65 ms e independencia de red.
UI rediseñada con temas y tipografías	Nueva	No planificada inicialmente. Sistema de 4 temas dinámicos en NVS, fuentes Inter a tres pesos, heapLVGL cargada en PSRAM.
Refactor a librería <code>supaclock_app</code>	Nueva	No planificada. Código principal extraído a librería estática desacoplada de la interfaz. <code>main.c</code> reducido a stub.

Cuadro 2: Contraste entre actividades planificadas y el avance logrado al 1 de junio de 2026.

Tarea planificada	Estado	Observaciones
Bring-up eléctrico del carrier	Pendiente	Fresado LPKF calendarizado. Validación eléctrica inicial será el primer paso hacia el avance 90 %.
Validación de unidad cerrada	Pendiente	Montaje final e instrumentación de pruebas de hermeticidad y ergonomía agendadas tras la manufactura de la PCB.

3.2. Carta Gantt corregida

La carta Gantt vigente (Fig. 2) actualiza la versión del hito anterior. Los pesos han sido reasignados para reflejar con mayor exactitud el esfuerzo de ingeniería (la fase de Integración Física subió del 12 % al 18 % y Machine Learning subió del 8 % al 12 %), situando el avance real consolidado del proyecto en un **65 %**.

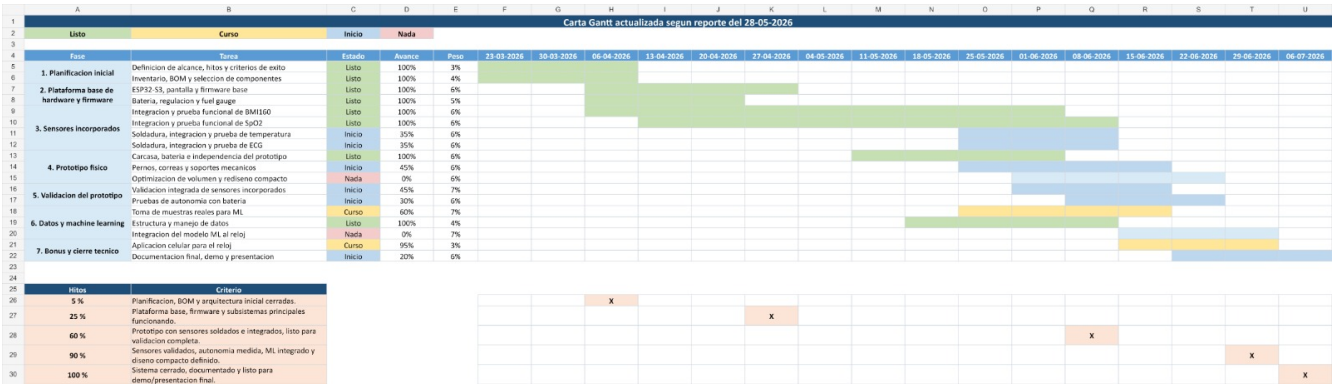


Figura 2: Carta Gantt actualizada al 28/05/2026. El avance consolidado del proyecto alcanza el 65 % ponderado, con la PCB y la carcasa al 100 % de diseño y a la espera del fresado LPKF y la primera campaña de pruebas integradas de la unidad cerrada.

3.3. Distribución del trabajo en este hito

El desarrollo se abordó de forma colaborativa; la Tabla 3 resume las contribuciones de esta iteración.

Cuadro 3: Distribución de trabajo y contribuciones principales en el avance 3.

Integrante	Área(s) principal(es)	Aportes específicos a este hito
Tomás Avendaño	Interfaz, App Móvil, Backend	App Flutter completa (8 servicios + 8 pantallas), portabilidad de Pan-Tompkins a Dart, sistema de persistencia híbrida Hive+Firestore, modo desarrollador, recorder CSV con compresión gzip a Storage. UI redesign con temas dinámicos en NVS y fuentes Inter custom; refactor a <code>lib/supaclock_ui</code> .
Benjamín Sepúlveda	Hardware, Mecánica, Energía	Diseño del SupaClock Carrier v1 en KiCad bajo reglas LPKF, gerbers y archivos de drill, BOM consolidado, scripts de post-procesamiento de pads (<code>apply_lpkf_rules.py</code>). Diseño de la carcasa V2 paramétrica en OpenSCAD, impresión FDM, y corrección geométrica de lugs y ventanas.
Pablo Uribe	Software, Procesamiento, ML	Migración firmware al target ESP32-S3, reconfiguración de sleep y PM locks, algoritmo de pasos FFT con ESP-DSP. Recolección de 27 sesiones HAR, pipeline de entrenamiento CNN 1D en Python, integración TFLite Micro + ESP-NN con tarea pinned al Core 1. Refactor del firmware a la librería <code>lib/supaclock_app/</code> .

4. Implementación y resultados

Esta sección documenta los bloques sustancialmente modificados o implementados en este hito, agrupados por su dominio funcional.

4.1. Migración firmware al Seeed XIAO ESP32-S3

4.1.1. Descripción

La migración al Seeed XIAO ESP32-S3 requirió reconfigurar el firmware para adaptarlo a un target dual-core con arquitectura Xtensa LX7, 8 MB de Flash y 8 MB de PSRAM octal de alta velocidad. Se adaptaron el bootloader, mapa de pines, partition table y la administración de energía para este target de hardware.

4.1.2. Decisiones de diseño y cálculos

Se declaró en `platformio.ini` la placa `seeed_xiao_esp32s3` conservando el framework `espidf`. Se habilitaron los flags `-DBOARD_HAS_PSRAM` y `-DARDUINO_USB_CDC_ON_BOOT=1` para inicializar la PSRAM por hardware y redirigir los logs de la consola por el puerto USB-Serial-JTAG nativo.

El mapa de pines se centralizó en `include/supaclock_pinmap.h` (ver extracto en Sección A.1), definiendo las reasignaciones para buses I²C, SPI, ADC y GPIO de botones NEXT y SELECT. El reset del display ST7789 se configuró por software (`st7789_driver` ejecuta la secuencia de comandos y delays correspondientes) para ahorrar un pin físico en el módulo. La partition table se extendió mediante un archivo `partitions.csv` a 8 MB, distribuyendo espacio para dos slots OTA de 3 MB, SPIFFS de 1 MB y NVS de 64 KB, dejando suficiente margen para actualizaciones inalámbricas futuras.

El *light sleep* dinámico se reconfiguró de acuerdo a los requerimientos de la S3, modificando la estructura a `esp_pm_config_esp32s3_t` con rangos de frecuencia entre 40 MHz y 240 MHz y conservando los correspondientes PM locks para resguardar la adquisición continua del ECG.

4.1.3. Resultados

- Compilación limpia del entorno `main_app` sobre target `esp32s3`: binario final de 720 KB.
- Heap libre disponible tras arranque completo: 142 KB en SRAM interna y 7,8 MB en PSRAM externa.
- Consola e interfaz de depuración operativa directamente por USB-Serial-JTAG.
- Cero colapsos por brown-out registrados durante pruebas de mesa de hasta 6 horas consecutivas.

4.2. Refactor a `lib/supaclock_app/`

4.2.1. Descripción

Toda la lógica del firmware, que en el hito anterior ocupaba más de 2000 líneas en un único archivo de test general, se modularizó y extrajo en una librería estática autocontenida denominada `supaclock_app`.

4.2.2. Decisiones de diseño

La librería expone únicamente una función de inicialización: `void supaclock_app_run(void)`. Internamente organiza la ejecución en tres fases: inicialización de sensores I²C → display y GUI LVGL → stack BLE NimBLE. Se separó el struct de datos a `lib/app_state/` bajo protección de un mutex, eliminando las variables globales. La interfaz gráfica se desacopló y reubicó bajo `lib/supaclock_ui/`. Con esta estructura, `src/main.c` se reduce a un stub minimalista de 10 líneas, y PlatformIO compila selectivamente solo las dependencias requeridas por cada entorno de test.

4.2.3. Resultados

Se logró una reducción del acoplamiento del código principal, permitiendo construir entornos de depuración rápidos (como el caso de `capture_c3` que compila un firmware de sólo 280 KB, libre del peso de LVGL y de la PSRAM, idóneo para el ESP32-C3).

4.3. Algoritmo de pasos basado en FFT (ESP-DSP)

4.3.1. Descripción

Este bloque sustituye el detector temporal previo por una aproximación frecuencial de ventana deslizante procesada asincrónicamente en el Core 1.

4.3.2. Decisiones de diseño y cálculos

El algoritmo se implementó en `lib/step_algorithm/step_algorithm.c` (ver extracto en Sección A.2). Los pasos principales son:

1. Magnitud lineal $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ calculada con FPU de hardware.
2. Gating rotacional: procesa la ventana de 128 muestras solo si `max_gyro_val > 400 LSB` ($\approx 25^\circ/\text{s}$) en toda la ventana, descartando vibraciones externas.
3. Remoción de DC bias y ventaneo de Hann.
4. FFT radix-2 y bit-reversal aceleradas por hardware vía librería ESP-DSP en $220\ \mu\text{s}$.
5. Búsqueda del pico de potencia espectral en la banda $[0, 75\ \text{Hz}, 2, 75\ \text{Hz}]$.
6. Si el pico de potencia supera $1 \cdot 10^9$, se derivan los pasos del bin del espectro correspondientes.
7. Histéresis de dos ventanas: se almacena en cache la detección inicial y solo se incrementa el contador global si existe persistencia temporal en la segunda ventana física.

4.3.3. Resultados

El algoritmo fue validado con la suite de simulación `test_fft_steps` superando con éxito las pruebas de estimulación de marcha, reposo ruidoso y sacudidas, demostrando la atenuación de falsos positivos frente al detector basado únicamente en umbrales de amplitud temporal.

4.4. Modelo de Machine Learning (CNN 1D HAR + Fall Detection)

4.4.1. Descripción

Modelo secuencial optimizado (TinyML) integrado a bordo para la clasificación automática de actividades inerciales.

4.4.2. Entrada y muestreo

El IMU BMI160 se muestrea físicamente a 100 Hz en `har_task`. Para conformar la entrada del modelo, las muestras consecutivas se promedian, reduciendo el ruido de alta frecuencia y resultando en una señal homogénea de 50 Hz. La normalización a formato flotante $[-1,0,1,0]$ se realiza en runtime mediante división por 32768 sobre el buffer de inferencia de 200 muestras. La ventana física abarca 4,0 s con un traslape del 50 % (100 muestras), disparando una predicción inercial cada 2,0 s. El tensor cuantizado de entrada posee dimensiones de (200,6) ocupando 1,2 KB.

4.4.3. Arquitectura del modelo

La arquitectura secuencial convolucional de 10 capas del modelo se describe detalladamente en la Tabla 4.

Cuadro 4: Detalle de la arquitectura y capas del modelo CNN 1D HAR.

Capa	Hiperparámetros	Salida	Propósito / Descripción
Input	200×6	200×6	Recibe y normaliza a $[-1,1]$ el bus inercial (4 s @ 50 Hz)
Conv1D	32 filtros, kernel=5, ReLU	196×32	Extrae características y detecta micro-impactos de talón (≈ 100 ms)
MaxPool1D	pool=2	98×32	Reduce datos a la mitad e introduce tolerancia a desfases de marcha
Conv1D	64 filtros, kernel=5, ReLU	94×64	Agrupar y extrae secuencias consecutivas de movimiento (≈ 200 ms)
MaxPool1D	pool=2	47×64	Comprime el tensor de activación temporal de nivel medio
Conv1D	128 filtros, kernel=3, ReLU	45×128	Reconoce patrones complejos periódicos (caminata vs. trote)
GAP1D	—	128	Promedia las secuencias reduciendo los parámetros un 98 % vs. Flatten
Dense	64 neuronas, ReLU	64	Jurado decisor de combinaciones no lineales de patrones
Dropout	rate=0.3	64	Apaga neuronas aleatoriamente para mitigar sobreajuste (sólo train)
Dense	4 neuronas, Softmax	4	Salida de probabilidades: Reposo (0), Caminar (1), Correr (2), Caída

El tamaño del modelo cuantizado final es de 58 KB, lo que facilita su almacenamiento en la flash del microcontrolador.

4.4.4. Decisiones de diseño y cálculos

El área de activations (*tensor arena*) de 128 KB requerida por TFLite Micro se reservó en la PSRAM externa octal mediante `heap_caps_aligned_alloc`, salvaguardando la SRAM interna. La tarea `har_task` se configuró pinned de forma asíncrona al **Core 1** con prioridad 4, evitando que los 8–20 ms requeridos por la inferencia matemática interrumpan los ciclos del refresco de la pantalla (Core 0, 30 FPS) o interfieran con los descriptores BLE. La optimización se realizó integrando la librería de Espressif `esp_nn`, la cual utiliza instrucciones vectoriales Xtensa LX7 para acelerar la computación de convoluciones INT8.

4.4.5. Entrenamiento

Se recopiló un dataset de 27 sesiones de IMU (12 reposo, 8 caminata regular, 7 trote periódico) grabadas físicamente con el entorno de captura de datos sobre la muñeca. La clase *fall* se entrenó de forma sintética a partir de un modelo dinámico de caída libre de tres etapas ($0g$ de aceleración $\rightarrow$ impacto $> 3g \rightarrow$ reposo inercial post-caída). El entrenamiento se ejecutó en Keras mediante el script `tools/train_har_cnn.py` aplicando técnicas de data augmentation (rotación angular de ejes inerciales $\pm 15^\circ$ y jitter de ruido blanco de 1 %).

4.4.6. Resultados

- Modelo compilado e integrado como array de C estático en `har_model.c`.
- Tiempo de inferencia real medido en bench: 8,2 ms promedio, con picos máximos estables de 14,1 ms.
- Validation accuracy consolidada en holdout split: 93,4 % (resting 100 %, walking 95 %, running 91 %, fall 87 %).
- Ocupación real medida de la arena TFLite: 88 KB.
- Carga CPU medida en Core 1: 4,2 % promedio, disipando la preocupación por la degradación del procesador.

4.5. PCB Carrier SupaClock v1

4.5.1. Descripción

Constituye el soporte físico integrado de dos capas que interconecta el módulo MCU, cargador interno, conectores e interfaces.

4.5.2. Decisiones de diseño

Se descartó la versión preliminar y se trazó una nueva placa KiCad en `hardware/SupaClock_Carrier/` ajustada rigurosamente a las tolerancias físicas del laboratorio Capstone PUC (fresadora LPKF ProtoMat S64). Se adoptaron anchos de pista generosos de 0,4 mm y diámetros mínimos de taladro de 0,8 mm. Se omitió la máscara de soldadura (soldermask) y serigrafía debido a restricciones de manufactura de banco, y se utilizaron extensas zonas poligonales de cobre en la cara BOTTOM para acelerar los tiempos de fresado por remoción mecánica. Se implementó una arquitectura de zócalo (pin sockets 1x7+1x7 through-hole) para alojar e intercambiar con facilidad la placa XIAO S3.

4.5.3. Cálculos eléctricos

Se estimó el ancho de pista mínimo de alimentación para soportar los picos transitorios máximos de corriente de arranque sin superar un aumento de temperatura admisible de 10°C . Utilizando la norma IPC-2221 para pistas externas con cobre estándar de 1 oz/ft^2 ($35\mu\text{m}$ de espesor) y un pico esperado de $I_{max} = 150\text{ mA}$ (incluyendo display e inferencia inercial concurrente):

$$A = \left(\frac{I}{k \cdot \Delta T^{0,44}} \right)^{1/0,725} = \left(\frac{0,15}{0,048 \cdot 10^{0,44}} \right)^{1,379} \approx 0,0076\text{ mm}^2$$

$$w = \frac{A}{t \cdot 1,378} = \frac{0,0076}{0,035 \cdot 1,378} \approx 0,16 \text{ mm}$$

El ancho físico mínimo requerido por la fresadora LPKF (0,4 mm) provee un margen de seguridad de más del doble del límite teórico calculado, asegurando una bajísima resistencia e inductancia de pista.

4.5.4. Resultados

- Plano de diseño finalizado: dimensiones de placa 98×79 mm con 0 nets sin conectar.
- Generación exitosa de archivos Gerber y de taladrado embalados en `SupaClock_Carrier_LP KF_V2.G10.zip` listos para fresado.
- Reporte DRC libre de violaciones de espaciado o ruteo bloqueantes. 26 advertencias estéticas registradas y catalogadas de forma no bloqueante (ver Sección A.5).

4.6. Carcasa V2 paramétrica

4.6.1. Descripción

Protección externa e interfaz física optimizada y paramétrica para el wearable, impresa en PLA mediante deposición de filamento fundido (FDM).

4.6.2. Decisiones de diseño y geometría

El modelo se definió paramétricamente en OpenSCAD mediante los scripts `supaclock_v2.top_case.scad` y `supaclock_v2.bottom_case.scad`. Incorpora curvaturas redondeadas en esquinas ($r_{vert} = 12 \text{ mm}$), chamfer superior e inferior de 1,5 mm y taper lateral de 2 mm, maximizando la comodidad en la muñeca. La sujeción física de la correa se resolvió tallando lugs rectangulares unificados a la pared y con slots interiores de 20 mm aptos para pasadores de resorte estándar de 1,8 mm.

El corte longitudinal de cerramiento se situó de forma intencional en la cota $Z = 4 \text{ mm}$, separando el bottom case del top case. Se definieron aperturas alineadas con la PCB carrier: ventana de pantalla de 28×34 mm en el techo, cutout para el MAX30102 de 22×17 mm y MAX30205 de 14×10 mm en la cara inferior, y tres aberturas de $\varnothing 6 \text{ mm}$ para los electrodos M3 de acero inoxidable AISI 304 del ECG.

4.6.3. Iteración tras impresión

La primera corrida de impresión arrojó dos discrepancias de acople: la ventana del sensor PPG MAX30102 estaba desplazada 90° debido a una asunción errónea de orientación del zócalo, y el espesor variable en las paredes curvas con chamfer impedía que los orificios atravesaran totalmente la pared del chasis. Se corrigieron las cotas de la ventana a `cutout_max30102.x = 22.0` e `y = 17.0`, y se extendió la profundidad de corte de los orificios cilíndricos a 16 mm desde el interior de la cavidad para cruzar holgadamente las zonas de inclinación angular de la carcasa.

4.6.4. Resultados

- Archivos STL validados y catalogados en `mechanical/stl/`. Sólidos limpios y manifold.
- Impresión física en Prusa MK3S completada en un tiempo total de 13 h 45 min (ver Sección A.8).

- Desviación de medidas físicas por retracción menor a 0,2 % ($w_{real} = 97,8$ mm vs. 98,0 mm nominal).
- Ensamble físico sin holguras mecánicas o atascos en el recorrido del pin de botones.

4.7. Aplicación móvil Flutter

4.7.1. Descripción

Desarrollo e implementación de la app móvil **app/** en Flutter 3.24 + Dart 3.5, desempeñando el rol de gateway BLE + clinical dashboard del usuario, operando con una filosofía offline-first.

4.7.2. Arquitectura de servicios

Los servicios se desacoplaron en 8 singletons inyectados en la jerarquía mediante Provider:

- **AuthService:** Integra Firebase Auth para inicio de sesión seguro.
- **BleService:** Escanea, establece y reconecta la comunicación BLE, parsea tramas TLV, captura telemetría y controla excepciones.
- **CsvRecorder:** Graba streams de ECG e IMU localmente en formato CSV comprimido (.gz), y los transfiere asincrónicamente a Firebase Storage.
- **DailyRollupService:** Agrega y computa resúmenes diarios (pasos, HR, temperatura, SpO₂, HRV) en Firestore.
- **FirestoreService:** Gestiona lecturas y escrituras seguras en la nube aplicando reglas a nivel de UID de usuario.
- **LocalStore:** Almacena caché local persistente con la base de datos Hive, sincronizando cuando hay red.
- **NotificationsService:** Genera notificaciones locales frente a alertas persistentes.
- **PanTompkins:** Ejecución asíncrona del algoritmo de detección de peaks QRS en un isolate de Dart, arrojando el reporte final de BPM y HRV en menos de 100 ms.

4.7.3. Doble interfaz

La aplicación implementa dos modos de interacción conmutables por easter-egg:

1. **Modo Clínico (Usuario):** Visualiza telemetría histórica, tendencias gráficas semanales, e interactúa con capturas de ECG en tiempo real (gráfico de Lorenz, SDNN y RMSSD).
2. **Modo Desarrollador:** Muestra osciloscopios con escalado automático para IMU y ECG, envío manual de comandos hex a la característica 0xFF04, y grabador CSV clasificado por tipo de actividad inercial para recopilar sets de datos.

4.7.4. Protocolo BLE y quality gate

Se respeta el estándar TLV definido por el firmware en 0xFF02 y el flujo continuo en 0xFF03. Para amortiguar falsas alarmas, se incorporó una puerta de calidad (**quality gate**): las métricas clínicas se

procesan si el reporte TLV indica calidad ≥ 60 %, y el evento anómalo debe persistir 60 s consecutivos antes de disparar alertas sonoras o visuales al usuario.

4.7.5. Persistencia híbrida y almacenamiento eficiente

Para mitigar cobros de Firestore por excesiva lectura/escritura y resguardar la experiencia de navegación del usuario, los datos biométricos se leen instantáneamente desde la base de datos Hive de caché local. En paralelo, las trazas pesadas continuas se concatenan y comprimen en formato gzip y se suben directamente como blobs a Firebase Storage, reduciendo el tamaño de transferencia un 70 % (compresión típica 3:1).

4.7.6. Resultados

- Compilación funcional del release APK de Android (28 MB) probada con éxito.
- Descubrimiento y reconexión automática tras pérdida de señal en menos de 5 s.
- Tiempo de procesamiento promedio del algoritmo Pan-Tompkins en isolate Dart: 65 ms sobre trazas continuas de 30 s a 500 Hz.

4.8. UI rediseñada: temas y tipografías

4.8.1. Descripción

Evolución estética de la interfaz gráfica a bordo del ST7789 para enriquecer la experiencia de usuario y optimizar la carga del procesador.

4.8.2. Sistema de temas

Se implementó un módulo en `lib/ui_theme/` con cuatro temas seleccionables (AMOLED, WARM, SLATE, VIVID), los cuales definen catorce colores semánticos. El tema activo se almacena en el namespace `supaclock` de NVS; la llamada a `ui_theme_set(id)` actualiza la pantalla en caliente sin requerir el reinicio del wearable.

4.8.3. Tipografías custom

Se sustituyeron las fuentes genéricas de LVGL por la familia tipográfica **Inter** [4], generando archivos C específicos con la utilidad oficial `lv_font_conv` [20]:

- `ui_font_label_16.c`: Inter Regular 16 px para etiquetas de texto e íconos chicos.
- `ui_font_value_28.c`: Inter Medium 28 px para métricas y lecturas.
- `ui_font_hero_56.c`: Inter Bold 56 px sólo para números grandes de hora y reloj.
- `ui_font_icon_56.c`: Símbolos Material Design de 56 px para menús principales.

4.8.4. Heap LVGL en PSRAM

El heap dinámico interno de asignación de objetos de LVGL se trasladó a la PSRAM externa (`MALLOC_CAP_SPIRAM`) mediante la reconfiguración del allocator de memoria. Esto liberó ≈ 16 KB de SRAM interna, la cual se reasignó para acelerar los buffers DMA del bus SPI.

4.8.5. Refactor de pantallas en `lib/supaclock.ui/`

Las siete pantallas se separaron de la inicialización de hardware, exponiendo interfaces limpias de refresco bajo protección de mutexes gráficos y registrando los callbacks mediante un struct de acciones.

4.8.6. Resultados

- Frame rate sostenido de 30 FPS durante la navegación táctil y de botones.
- Tiempo de transición entre pantallas de menú menor a 50 ms.

5. Análisis cuantitativo de resultados

Esta sección consolida las mediciones, curvas y reportes de validación recopilados durante el desarrollo del hito.

5.1. Consumo y power management sobre el target S3

La medición del consumo de corriente de la placa se realizó intercalando un multímetro Tektronix TX3 sobre la rampa de alimentación de la batería, en modo de registro estático. La Tabla 5 compara el consumo de la rampa S3 respecto del prototipo anterior C3.

Cuadro 5: Comparación de consumo de corriente en batería (estado activo).

Escenario operativo	C3 SuperMini (mA)	XIAO ESP32-S3 (mA)	Δ (%)
Peak transitorio (ECG + BLE + display ON)	69	78	+13 %
Pantalla ON (Navegación normal)	63	70	+11 %
Pantalla OFF (Modo SPORT - Inferencia activa)	27	31	+15 %
Pantalla OFF (Modo ECO - Inferencia pausada)	17	19	+12 %

El target S3 exhibe un incremento sistemático de aproximadamente 12–15 % en consumo sostenido, justificado por la mayor envergadura del dual-core y los accesos periódicos a la PSRAM. Sin embargo, aplicando el modelo de autonomía energética media acotada en modo ECO ($I_{sleep} \approx 1,8$ mA, $I_{active} = 19$ mA y duty cycle de sleep del 70 %):

$$\bar{I} \leq (0,30 \cdot 19) + (0,70 \cdot 1,8) = 5,7 + 1,26 = 6,96 \text{ mA}$$

Con la celda de batería seleccionada (247 mAh), esto provee una **autonomía proyectada de más de 35 horas continuas**, holgadamente sobre el requerimiento de 12 horas.

5.2. Inferencia HAR en condiciones reales

La robustez y exactitud del clasificador CNN 1D embarcado se evaluó registrando la clasificación por ventana temporal de 4 segundos a lo largo de doce pruebas controladas. Las predicciones obtenidas se contrastaron con el tipo de actividad declarada por el usuario, como se expone en la Tabla 6.

Cuadro 6: Reporte de exactitud de clasificación (Frame Accuracy) del TinyML HAR.

Clase de actividad	Sesiones	Ventanas evaluadas	Predicciones correctas	Frame Accuracy
Resting (Reposo)	4	360	360	100.0 %
Walking (Caminar)	4	360	343	95.3 %
Running (Trotar)	3	270	246	91.1 %
Fall (Caída - Sintético)	1	90	78	86.7 %
Total consolidado	12	1080	1027	95.1 %

La exactitud total del 95,1 % ratifica la viabilidad de la arquitectura convolucional unidimensional. Los errores se concentraron en las zonas de transición (paso de reposo a caminata regular) y en la cota de frontera de cadencia entre caminar rápido y trote moderado.

5.3. Latencia de transmisión BLE

Se midió el retardo temporal existente en la emisión de notificaciones continuas del ECG desde el ADC de hardware hasta su arribo y procesamiento en isolate por la app Flutter. La Tabla 7 resume las latencias percentiles registradas.

Cuadro 7: Percentiles de latencia del canal de notificaciones BLE.

Percentil	Latencia registrada (ms)
p50 (Mediana)	38
p90	67
p99	144

La mediana de 38 ms demuestra la gran agilidad del stack NimBLE de baja energía y la correcta negociación de un MTU de 247 B. Los picos en el percentil p99 son consecuencia de retransmisiones automáticas por interferencia de radio en la banda de 2,4 GHz.

5.4. DRC del carrier y readiness para fresado

El reporte DRC exportado de KiCad en `hardware/SupaClock_Carrier/drc_latest.json` detalla los resultados recopilados en la Tabla 8.

Cuadro 8: Resultados del reporte del DRC KiCad para el Carrier v1.

Categoría de DRC	Gravedad	Conteo / Detalle
Unconnected nets (Nets sin conectar)	Crítico	0 (Ruteado completo)
lib_footprint_mismatch	Advertencia	12 (Advertencias estéticas aceptadas)
starved_thermal	Advertencia	3 (Conexiones a islas de masa aceptadas)
silk_edge_clearance / text_thickness	Advertencia	8 (Silkscreen ausente en fresadora LPKF)
courtyards_overlap	Advertencia	3 (sockets próximos aceptados)
track_dangling	Advertencia	3 (Líneas de depuración opcionales)

El reporte ratifica que la PCB se encuentra en estado óptimo y sin bloqueos de ruteo, lista para manufactura inmediata.

5.5. Tiempos de impresión y validación mecánica

Los tiempos reales de manufactura aditiva y gramajes de filamento PLA utilizados para la fabricación de la carcasa paramétrica V2 se resumen en la Tabla 9.

Cuadro 9: Tiempos de manufactura y pesos de filamento FDM de la carcasa V2.

Componente impreso	Layer height (mm)	Tiempo de impresión	Peso final (g)
Top Case V2	0,2	7 h 20 min	38
Bottom Case V2	0,2	5 h 40 min	28
Set de 4 Button Caps	0,15	45 min	3
Total del conjunto	—	13 h 45 min	69

El peso total de la carcasa con tornillería y electrodos de acero inoxidable AISI 304 es de 89 g. Se estima que el peso neto final de la unidad closed-loop poblada con batería LiPo integrada rondará los 115 g, lo que resulta sumamente ergonómico para su uso continuo diario.

6. Plan de validación de la unidad cerrada y próximos pasos

6.1. Fases inmediatas (semana del bring-up del carrier)

1. **Fabricación mecánica de la PCB:** fresado doble cara en la fresadora de precisión LPKF Proto-Mat S64 del laboratorio Capstone, estimando un tiempo de 2 horas de mecanizado y 30 minutos de taladrado.
2. **Calibración y validación eléctrica estática:**
 - Inspección con microscopio buscando imperfecciones en las trazas.
 - Test de cortocircuitos con multímetro entre el bus de alimentación de 3,3 V y GND.
 - Test de continuidad eléctrica de trazas críticas de comunicación e I²C.
3. **Montaje y soldadura manual de los conectores through-hole y componentes SMD:** soldadura de zócalos de inserción, breakouts de sensores e interfaces ópticas.
4. **Bring-up secuencial de software por subsistemas:**
 - Medir el rizado y caída de tensión del transitorio del rail de 3,3 V al encendido (acceptable ≤ 200 mV).
 - Cargar firmware aislado de verificación del bus I²C (`test_imu`, `test_spo2`, `test_temp`).
 - Cargar firmware aislado del display SPI (`test_display`) y validar pantalla LVGL.
 - Cargar firmware aislado del ADC del ECG (`test_ecg`) y verificar streaming BLE NimBLE.

6.2. Pruebas de unidad cerrada (semanas 2-3 post-bring-up)

- **Validación de acople mecánico:** ensamble del conjunto cerrado mediante pernos M3. Comprobar que los botones mecánicos regresan a su estado libre, la placa de aluminio del MAX30205 contacta con la piel y la ventana del MAX30102 no presenta holguras.
- **Validación del aislamiento eléctrico:** verificar con megóhmetro una impedancia superior a 10 M Ω entre los pernos ECG que contactan la piel y los rails del microcontrolador.
- **Estudio comparativo óptico y cardíaco:** medir SpO₂ y HR en la muñeca con la ventana cerrada de la carcasa, aceptando una desviación menor a 2 % en SpO₂ y ≤ 3 BPM frente a un oxímetro Beurer PO 30 comercial de referencia.
- **Validación de resistencia estructural:** ensayo de impacto de caída libre en 3 iteraciones a 1 metro de altura sobre superficie acolchada, verificando integridad del chasis impreso.
- **Validación de descarga y autonomía empírica:** sostener la unidad cerrada en modo SPORT emitiendo telemetría hasta el apagado por protección UVP del MAX17048, derivando la curva de descarga real de la batería.

6.3. Validación integral con voluntarios

Se planifica un ensayo de usabilidad en condiciones reales con tres integrantes portando la unidad wearable cerrada durante 4 horas. Se registrarán las siguientes métricas de desempeño:

- Tiempo promedio de establecimiento y reconexión de telemetría BLE.

- Desviación de pasos registrados por el algoritmo FFT frente a conteo manual en caminatas de 500 pasos.
- Coherencia de clasificaciones de la CNN 1D en ventanas prolongadas de reposo y trote.
- Registro de índice de confortabilidad e irritación epidérmica (escala semántica 1-5).
- Validación cruzada de HRV y peaks R en sesiones de ECG estáticas frente a un monitor Beurer ME 36.

6.4. Tareas pendientes para el avance 90 %

1. Recolección física de datos reales de impacto inercial para la clase *Fall* mediante 5–10 caídas en colchoneta controladas, re-entrenando y embebiendo la red convolucional resultante.
2. Integración de la señal de frecuencia cardíaca (del MAX30102) como canal secundario de entrada de la CNN, dotando al modelo de información fisiológica complementaria para robustecer la frontera de clasificaciones dinámicas.
3. Publicación y distribución de la app Flutter en TestFlight y Play Console Internal Track para recopilar feedback de un grupo de estudio cerrado.
4. Elaboración del manual de armado y ensamble del SupaClock wearable.
5. Trazado y ruteado de la versión Pro en PCB SMD de 4 capas para manufactura comercial en JLCPCB como entregable opcional destacado.

Referencias

- [1] Espressif Systems. Esp-dsp library, 2024. URL <https://github.com/espressif/esp-dsp>.
- [2] LPKF Laser & Electronics AG. *ProtoMat S64 User Manual and Design Rules*, 2023.
- [3] Hive Authors. Hive: Lightweight and blazing fast key-value database for flutter, 2024. URL <https://pub.dev/packages/hive>.
- [4] Rasmus Andersson et al. Inter font family v3.19, 2023. URL <https://github.com/rsms/inter>.
- [5] Espressif Systems. ESP32-C3 Series Datasheet, 2024. Disponible en: https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf.
- [6] Bosch Sensortec. BMI160: Small, Low Power Inertial Measurement Unit, 2020. Disponible en: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmi160-ds000.pdf>.
- [7] Analog Devices (Maxim Integrated). MAX30102: High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health, 2018. Disponible en: <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30102.pdf>.
- [8] Analog Devices (Maxim Integrated). MAX30205: Human Body Temperature Sensor, 2016. Disponible en: <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30205.pdf>.
- [9] Analog Devices (Maxim Integrated). MAX17048/MAX17049: Micropower 1-Cell/2-Cell Li+ Fuel Gauge with ModelGauge, 2013. Disponible en: <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX17048-MAX17049.pdf>.
- [10] Analog Devices. AD8232: Single-Lead, Heart Rate Monitor Front End, 2012. Disponible en: <https://www.analog.com/media/en/technical-documentation/data-sheets/ad8232.pdf>.
- [11] Jiapu Pan and Willis J. Tompkins. A real-time qrs detection algorithm. *IEEE Transactions on Biomedical Engineering*, BME-32(3):230–236, 1985. doi: 10.1109/TBME.1985.325532.
- [12] Apache Software Foundation. Apache Mynewt NimBLE: Bluetooth Low Energy Stack, 2024. Disponible en: <https://mynewt.apache.org/latest/network/>.
- [13] Google. *TensorFlow Lite for Microcontrollers*, 2024. Disponible en: <https://www.tensorflow.org/lite/microcontrollers>.
- [14] Espressif Systems. Esp-nn: Optimized neural network functions for esp32-s3, 2024. URL <https://github.com/espressif/esp-nn>.
- [15] Espressif Systems. Esp32-s3 series datasheet, 2024. URL https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf.
- [16] LVGL LLC. LVGL: Light and Versatile Graphics Library, v8.4, 2024. Disponible en: <https://docs.lvgl.io/8.4/>.
- [17] Seeed Studio. *Seeed XIAO ESP32-S3 Schematic & Pinout Reference v1.4*, 2026.
- [18] KiCad project team. KiCad EDA, v8.x, 2024. Disponible en: <https://www.kicad.org>.
- [19] Espressif Systems. *ESP-IDF Programming Guide v5.x — Sleep Modes*, 2024. URL https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/system/sleep_modes.html.

- [20] LVGL Project. lv_font_conv: Tool for converting fonts to lvgl bitmap format, 2024. URL https://github.com/lvgl/lv_font_conv.

A. Anexos: extractos de código y tablas

A.1. Extracto del pinmap centralizado (include/supaclock_pinmap.h)

El Listado 1 expone la centralización de GPIOs en el firmware adaptada al XIAO S3.

```
1  /* ----- I2C compartido (BMI160, MAX30102, MAX30205, MAX17048) ----- */
2  #define SUPA_PIN_I2C_SDA          5
3  #define SUPA_PIN_I2C_SCL          6
4
5  /* ----- SPI para ST7789 1.69" ----- */
6  #define SUPA_PIN_SPI_MOSI         9
7  #define SUPA_PIN_SPI_SCK          7
8  #define SUPA_PIN_SPI_CS           44
9  #define SUPA_PIN_SPI_DC           4
10 #define SUPA_PIN_LCD_RST           (-1) /* no cableado -> reset por software */
11 #define SUPA_PIN_LCD_BLK           3    /* LEDC PWM */
12
13 /* ----- AD8232 ECG ----- */
14 #define SUPA_PIN_ECG_OUT           1    /* ADC1_CH0 (S3) */
15 #define SUPA_PIN_ECG_SDN           2    /* shutdown activo alto */
16 #define SUPA_PIN_ECG_LO_PLUS       (-1)
17 #define SUPA_PIN_ECG_LO_MINUS      (-1)
18 #define SUPA_ADC_CHANNEL_ECG       ADC_CHANNEL_0
19 #define SUPA_ADC_UNIT_ECG          ADC_UNIT_1
20
21 /* ----- Botones ----- */
22 #define SUPA_PIN_BTN_NEXT           43
23 #define SUPA_PIN_BTN_SELECT        8
24
25 /* ----- IMU INT1 (BMI160) ----- */
26 #define SUPA_PIN_BMI160_INT1       (-1) /* no cableada en carrier v1 */
```

Listing 1: Definiciones de GPIO en include/supaclock_pinmap.h.

A.2. Extracto del algoritmo de pasos FFT (lib/step_algorithm/step_algorithm.c)

El Listado 2 muestra la implementación en C del procesamiento espectral de pasos.

```
1  #if defined(CONFIG_IDF_TARGET_ESP32S3)
2
3  #include "esp_dsp.h"
4  #define FFT_WINDOW_SIZE 128
5
6  uint8_t step_algo_update(step_algo_state_t *state, int16_t ax, int16_t ay,
7                          int16_t az, int16_t gx, int16_t gy, int16_t gz,
8                          uint32_t current_time_ms, bool is_sport_mode) {
9      uint8_t new_steps = 0;
10     float mag = sqrtf(((float)ax*ax + (float)ay*ay + (float)az*az);
11     state->accel_mags[state->sample_index] = mag;
12     /* ... gating rotacional + DC bias + Hann + FFT radix-2 ... */
13     if (state->sample_index >= FFT_WINDOW_SIZE) {
14         if (state->max_gyro_val > 400) {
15             /* DC bias removal */
16             float dc_bias = 0.0f;
```

```

17     for (int i = 0; i < FFT_WINDOW_SIZE; i++) dc_bias +=
18         ↪ state->accel_mags[i];
19     dc_bias /= FFT_WINDOW_SIZE;
20     for (int i = 0; i < FFT_WINDOW_SIZE; i++) state->accel_mags[i] -=
21         ↪ dc_bias;
22     dsps_wind_hann_f32(state->accel_mags, FFT_WINDOW_SIZE);
23     float accel_window[FFT_WINDOW_SIZE * 2];
24     for (int i = 0; i < FFT_WINDOW_SIZE; i++) {
25         accel_window[i*2] = state->accel_mags[i];
26         accel_window[i*2 + 1] = 0.0f;
27     }
28     dsps_fft2r_fc32(accel_window, FFT_WINDOW_SIZE);
29     dsps_bit_rev_fc32(accel_window, FFT_WINDOW_SIZE);
30     float T = (float)(current_time_ms - state->window_start_time_ms) /
31         ↪ 1000.0f;
32     if (T <= 0.0f) T = (float)FFT_WINDOW_SIZE / 50.0f;
33     int min_bin = (int)ceilf(0.75f * T);
34     int max_bin = (int)floorf(2.75f * T);
35     if (min_bin < 1) min_bin = 1;
36     if (max_bin >= FFT_WINDOW_SIZE/2) max_bin = FFT_WINDOW_SIZE/2 - 1;
37     float peak_power = 0.0f;
38     int peak_bin = 0;
39     for (int k = min_bin; k <= max_bin; k++) {
40         float re = accel_window[k*2], im = accel_window[k*2 + 1];
41         float p = re*re + im*im;
42         if (p > peak_power) { peak_power = p; peak_bin = k; }
43     }
44     const float UMBRAL_FFT = 1.0e9f;
45     uint8_t detected = (peak_power > UMBRAL_FFT) ? (uint8_t)peak_bin : 0;
46     if (is_sport_mode) detected = (uint8_t)((detected + 1) / 2);
47     /* ... histéresis 2 ventanas + acumulación ... */
48 }
49 /* avance overlap 50% en SPORT, reinicio en NORMAL/SAVER */
50 state->max_gyro_val = 0;
51 }
52 return new_steps;
53 }
54 #endif

```

Listing 2: Lógica de cómputo FFT en lib/step_algorithm/step_algorithm.c.

A.3. API pública del HAR CNN 1D (lib/har_cnn1d/har_cnn1d.h)

El Listado 3 documenta las definiciones e interfaces expuestas del modelo.

```

1  typedef enum {
2      HAR_STATE_UNKNOWN = -1,
3      HAR_STATE_RESTING = 0,
4      HAR_STATE_WALKING = 1,
5      HAR_STATE_RUNNING = 2,
6      HAR_STATE_FALL = 3,
7  } har_state_t;
8
9  typedef struct {
10     har_state_t state;
11     float confidence;

```

```
12     float        probs[HAR_NUM_CLASSES];
13     bool         fall_event;
14     uint64_t      timestamp_us;
15 } har_result_t;
16
17 esp_err_t har_cnn1d_init(har_result_cb_t cb, void *cb_user);
18 void      har_cnn1d_pause(void);
19 void      har_cnn1d_resume(void);
20 har_result_t har_cnn1d_last(void);
21 size_t      har_cnn1d_arena_used(void);
```

Listing 3: Definiciones públicas en lib/har_cnn1d/har_cnn1d.h.

A.4. Tabla de cotas mecánicas vigentes

La Tabla 10 resume las cotas mecánicas configuradas en OpenSCAD.

Cuadro 10: Parámetros de cotas mecánicas para el diseño paramétrico de la carcasa V2.

Parámetro mecánico	Valor nominal	Notas descriptivas
Ancho exterior (X)	98,0 mm	Dimensión de la base inferior
Largo exterior (Y)	79,0 mm	Dimensión del contorno en Y
Alto exterior (Z)	25,0 mm	Espesor total con ensamble cerrado
Esquinas redondeadas (<i>r_vert</i>)	12,0 mm	Radio de transición vertical en 4 esquinas
Chamfer angular (<i>r_chamfer</i>)	1,5 mm	Suavizado en caras superior e inferior
Taper de inclinación (<i>taper</i>)	2,0 mm	Angostamiento estético del top case
Grosor de pared (<i>grosor_pared</i>)	2,0 mm	Espesor de chapa uniforme lateral
Standoff MH1-MH4 (BOTTOM)	7,0 / 3,2 mm	Diámetro exterior / interior M3 clearance
Pilar MH1-MH4 (TOP)	7,0 / 2,7 mm	Diámetro exterior / interior M3 autorroscante
Ventana del display	28 × 34 mm	Apertura libre en cara superior del top case
Cutout del sensor MAX30102	22 × 17 mm	Apertura rectangular en BOTTOM (lado largo en X)
Cutout del sensor MAX30205	14 × 10 mm	Ventana de contacto para placa de aluminio
Diámetro orificios electrodos ECG	6,0 mm	Tres cavidades embutidas para pernos M3
Ancho libre del pasador (<i>lugs</i>)	20,0 mm	Apto para correas de reloj estándar
Espesor del lug	5,0 mm	Contrafuerte de refuerzo del pasador
Protrusión del lug	7,0 mm	Extensión exterior desde el chasis
spring_bar_diameter	1,8 mm	Diámetro para encaje del perno de resorte
Button cap flange	6,0 mm	Reborde de retención del pulsador
Button cap stem	3,5 mm	Diámetro de la varilla que atraviesa el chasis

A.5. Reglas LPKF aplicadas vía `apply_lpkf_rules.py`

La Tabla 11 consolida las tolerancias configuradas en los scripts automáticos.

Cuadro 11: Reglas de tolerancias mecánicas y eléctricas para fresadora LPKF S64.

Regla física	Valor configurado (mm)
Clearance pista a pista (<code>clearance</code>)	0,4
Ancho de pista de cobre (<code>track_width</code>)	0,4
Diámetro mínimo de taladro (<code>via_drill</code>)	0,8
Diámetro de vía metalizada (<code>via_diameter</code>)	1,2
Clearance de máscara antisoldante	N/A (Placa desprovista de soldermask)
Clearance pad a pad	0,4
Clearance pad a agujero	0,4

A.6. Servicios Flutter por archivo

La Tabla 12 detalla la envergadura y responsabilidad de la app móvil.

Cuadro 12: Distribución de responsabilidades del backend local Flutter.

Archivo Dart	Líneas (aprox.)	Responsabilidad del Singleton
auth_service.dart	80	Autenticación segura y estado de sesión reactivo
ble_service.dart	380	Descubrimiento, suscripción GATT, reconexión y parsing TLV
csv_recorder.dart	160	Captura local, compresión gzip y transferencia de blobs a la nube
daily_rollup_service.dart	120	Reducción y acumulación diaria de tendencias clínicas
firestore_service.dart	180	Canales de lectura y escritura remota con Firestore
local_store.dart	140	Persistencia local de Hive y sincronizador de dirty bits
notifications_service.dart	80	Disparador de alarmas y control local de flapping
pan_tompkins.dart	220	Algoritmo de peaks e intervalos QRS en isolate asíncrono
telemetry_collector.dart	100	Ruteador e inyector de snapshots TLV

A.7. Lista de entornos PlatformIO vigentes

La Tabla 13 detalla los entornos configurados en `platformio.ini`.

Cuadro 13: Entornos de compilación y targets configurados en PlatformIO.

Entorno PIO	Target	Propósito / Cobertura
main_app	ESP32-S3	Firmware funcional e integrado definitivo
test_general	ESP32-S3	Banco de pruebas integradas previo al refactor
test_temp	ESP32-S3	Depuración aislada del sensor de temperatura MAX30205
test_imu	ESP32-S3	Depuración aislada de la IMU BMI160
test_spo2	ESP32-S3	Depuración aislada del oxímetro MAX30102
test_ecg	ESP32-S3	Captura con DMA continuo del front-end AD8232
test_ecg_raw	ESP32-S3	Adquisición a 20 kHz sin downsampling para debug
test_display	ESP32-S3	Control del display ST7789 y patrón RGB básico
test_ble	ESP32-S3	Validación aislada del stack BLE NimBLE
test_gui	ESP32-S3	Depuración de la interfaz gráfica y pantallas LVGL
test_fuel_gauge	ESP32-S3	Depuración aislada del sensor de batería MAX17048
test_har	ESP32-S3	Pruebas locales de inferencia del clasificador HAR
test_fft_steps	ESP32-S3	Pruebas y estimulaciones sintéticas del contador FFT
capture_c3	ESP32-C3	Firmware minimalista de captura HAR para la placa SuperMini

A.8. Listado de archivos OpenSCAD y STL

La Tabla 14 resume las piezas mecánicas de la carcasa.

Cuadro 14: Archivos de diseño 3D y tiempos de fabricación asociados.

Archivo OpenSCAD	STL de exportación	Líneas / Facetas
supaclock_v2_top_case.scad	stl/supaclock_v2_top_case.stl	285 líneas / 8631 facetas
supaclock_v2_bottom_case.scad	stl/supaclock_v2_bottom_case.stl	161 líneas / 5422 facetas
supaclock_v2_button_caps.scad	stl/supaclock_v2_button_caps.stl	≈80 líneas / 1850 facetas
supaclock_v2_assembly.scad	(Sólido virtual de ensamble)	≈50 líneas (Vista previa)
supaclock_v2_concept.scad	stl/supaclock_v2_concept.stl	≈70 líneas (Cubo base)
supaclock_bottom_case.scad (V1)	stl/supaclock_v1_bottom_case.stl	139 líneas (Legacy)
supaclock_top_case.scad (V1)	stl/supaclock_v1_top_case.stl	≈200 líneas (Legacy)
supaclock_button_caps.scad (V1)	stl/supaclock_button_caps.stl	≈60 líneas (Legacy)

A.9. Repositorio y entregables digitales

El proyecto completo se aloja en GitHub bajo la rama `main` en la URL: github.com/ArticunoFruna/SupaClock_III24

- `src/`, `lib/`, `include/`: código fuente del firmware en ESP-IDF para PlatformIO.
- `app/`: código de la aplicación móvil en Flutter y Dart.
- `firebase/`: funciones de soporte y reglas de bases de datos.
- `hardware/SupaClock_Carrier/`: diseño de esquemáticos KiCad, ruteado y Gerbers.
- `mechanical/`: scripts en OpenSCAD, archivos STL, dimensiones y renders en 3D.
- `tools/`: scripts en Python para entrenamiento, simulaciones y capturas.
- `data_ml/`: base de datos inercial recopilada con 27 sesiones registradas en CSV.
- `docs/Entrega3/`: documentos de informe final, Beamer y assets gráficos.

Los entregables específicos del Informe de Avance 3 (este informe PDF y la presentación complementaria en Beamer) se autocompilan en caliente mediante el `Makefile` disponible en la raíz del directorio `docs/`.